

PAOO L08 – L13 – activitate proiect

Contents

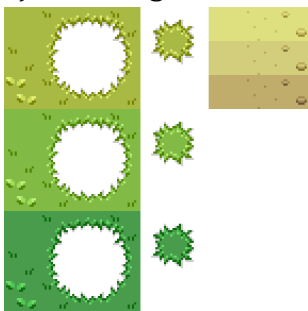
Crearea hărților	1
Tiles, Sprites, Tilemaps	1
Tilemap statice	2
Scrolling Tilemap	2
Straturi (layere)	3
Performanță	3
Crearea caracterelor	4
Deplasarea caracterelor	6
Coliziuni cu mediul sau alte obiecte	7
Bibliografie	14
Anexa 1	15

Crearea hărților

Tiles, Sprites, Tilemaps

Tilemap-urile sunt tehnici populare în dezvoltarea jocurilor, constând în construirea hărții nivelului din imagini mici, de formă regulată, numite plăci sau dale (**tiles**). Acest lucru duce creșterea performanței pentru că poate face ca încărcarea memoriei să fie minimă. Hărțile vor fi construite din imagini mici sau fragmente de imagini [1].

Cel cel mai eficient mod de a stoca imaginile este prin intermediul unui **spritesheet** în care toate dalele necesare sunt grupate într-un singur fișier de imagine. Când trebuie afișată o dală, doar o mică secțiune a acestei imagini este redată. Un exemplu de astfel de **tiles** și de utilizare se regăsește în imaginea de mai jos:



Fișierul spritesheetă[2]



Imagine posibilă în joc[2]

În mod natural fiecare dală poate fi identificată printr-un index (sau un identificator) ce poate fi creat/utilizat în momentul citirii obiectului din *spritesheet*.

Dalele pot fi mapate pe o matrice care indică harta unui nivel sau alte componente ale logicii jocului

ATENȚIE!

Pentru etapa a I-a a proiectului, aceasta matrice TREBUIE sa fie salvată în baza de date sau într-un fișier. Nu se va considera îndeplinită această condiție dacă harta este creată direct în cod.

Cea mai simplă definiție a tile-urilor într-un spritesheet este cea bazată pe forme pătrate NxN. Un caz generic ar fi dalele dreptunghiulare, dar acestea sunt mult mai puțin frecvente. Dalele pătrate permit două perspective[1]:

- De sus în jos (cum ar fi multe RPG-uri sau jocuri de strategie)
- Side-view (cum ar fi jocurile platformers, Super Mario Bros)

Tilemap statice

O hartă poate să se încadreze pe tot ecranul sau să fie mai mare și să fie vizualizată doar parțial. În primul caz, harta este statică (nu defilează pentru a fi afișată complet).

Randarea tilemap-urilor statice se poate face cu o buclă imbricată iterând peste coloane și rânduri. Un algoritm de randare ar putea fi:

```
for (column = 0; column < map.columns; column++) {  
    for (row = 0; row < map.rows; row++) {  
        tile = map.getTile(column, row);  
        x = column * map.tileSize;  
        y = row * map.tileSize;  
        drawTile(tile, x, y);  
    }  
}
```

Scrolling Tilemap

Scrolling tilemap-urile afișează doar o parte din universul jocului: se poate urmări un personaj sau o anumită regiune(zonă) a hărții la un moment dat.

În jocurile de acest tip este necesară să se facă o legătură între coordonatele lumii (poziția în care se află personajul urmărit de exemplu) și coordonatele ecranului (poziția reală în care aceste elemente sunt redată pe ecran). Coordonatele lumii pot fi exprimate în termeni de poziție a dalei (rând și coloană a hărții) sau în pixeli de pe hartă, în funcție de joc[1].

Pentru a putea transforma coordonatele lumii în coordonate de ecran, avem nevoie de coordonatele camerei, deoarece acestea determină ce secțiune a lumii este afișată[1]:

```
// presupunând camera raportată la colțul stânga sus:
```

```
function worldToScreen(x, y) {  
  // return { x - camera.x, y - camera.y };  
}
```

```
function screenToWorld(x, y) {  
  // return { x + camera.x, y + camera.y };  
}
```

Straturi (layere)

Grila vizuală este adesea alcătuită din mai multe straturi. Acest lucru permite o lume mai bogată, deoarece aceeași imagine poate fi utilizată cu fundaluri diferite. De exemplu, un copac care ar putea apărea deasupra mai multor tipuri de teren. Dacă personajele sau alte obiecte sunt desenate într-un layer aflat sub un altul, acest lucru poate permite efecte precum personaje care se plimbă în spatele copacilor sau clădirilor.

Performanță

Desenarea dalelor poate afecta performanța. De obicei, unele tehnici trebuie să fie puse în aplicare, astfel încât defilarea poate fi bună. O abordare este de a desena numai dalele care vor fi vizibile. O altă tehnică constă în pre-*randarea* hărții, atunci când este posibil, astfel încât plăcile nu trebuie să fie redesenat în fiecare cadru ci doar afișate sau ascunse atunci când este necesar. Desigur, dacă harta este mare, acest lucru nu este posibil. O metodă alternativă ar fi împărțirea hărții în secțiuni mari (cum ar fi o hartă completă împărțită în 10 x 10 părți de exemplu) iar la un moment dat să fie încărcată doar un astfel de segment în memorie.

În mod ideal, numai dalele care sunt vizibile ar trebui să fie *randate*.

Crearea hărților în jocuri poate duce la unele probleme legate de memorie dacă aceasta nu este gestionată corect. De aceea în practică se pot utiliza șabloane de proiectare ce ajută la rezolvarea unor situații nedorite. Printre acestea [șablonul Flyweight](#) poate fi util.

ATENȚIE!

Pentru etapa a II-a a proiectului, este important ca un obiect de tip *tile* să nu fie încărcat de mai multe ori în memorie ci să fie încărcat o singură dată și multiplicarea să fie realizată prin transmiterea referinței obiectului de tip *dală* creat sau printr-o altă metodă care nu consumă resurse. Se recomandă studierea [șablonul Flyweight](#).

Crearea caracterelor

Majoritatea jocurilor au în centru o serie de personaje. Acestea trebuie create, desenate, definit modul de deplasare în mediu, definite acțiunile sale, eventual o limită de viață sau abilități specifice.

De exemplu, **Bomberman** este protagonistul dintr-unul din cele mai cunoscute jocuri video este de obicei descris ca fiind tipul eroic, dar vesel, salvându-și deseori planeta natală de la un dezastru. El are capacitatea de a genera bombe.



Sau **Black Bomberman** arată identic/asemănător cu Bomberman și acționează ca al doilea jucător în modurile de poveste multiplayer.



Pentru a crea caracterele sunt necesare definirea unor proprietăți ale acestora, proprietăți ce trebuie ajustate în funcție de scopul jocului. Mai jos găsiți un exemplu de definire a unor astfel de attribute[1].

```
public abstract class Character extends Item
{
    public static final int DEFAULT_LIFE          = 10;
    /*!< Valoarea implicita a vietii unui caracter.*/
    public static final float DEFAULT_SPEED        = 3.0f;
    /*!< Viteza implicita a unu caracter.*/
    public static final int DEFAULT_CREATURE_WIDTH = 48;
    /*!< Latimea implicita a imaginii caracterului.*/
    public static final int DEFAULT_CREATURE_HEIGHT = 48;
    /*!< Inaltimea implicita a imaginii caracterului.*/

    protected int life;      /*!< Retine viata caracterului.*/
    protected float speed;   /*!< Retine viteza de deplasare caracterului.*/
    protected float xMove;   /*!< Retine noua pozitie a caracterului pe axa X.*/
    protected float yMove;   /*!< Retine noua pozitie a caracterului pe axa Y.*/

    /*
     * Alte proprietăți posibile ale caracterului
     */
    private String name;
    private String player class;
    private int experience;
    private int level;
    private int gold;
    private int strength;
    private int agility;
    private int resilience;
    private int deftness;
    private int charm;
    private int magical mending;
    private int magical might;

    /*
     * Bag object instantiation
     */
    private Bag b;

    //.....
```

Unui caracter i se pot adăuga valori diferite ale proprietăților în funcție de tipul acestuia. Pentru aceasta se pot crea metode speciale. Utilizarea unor șabloane de proiectare pot fi de asemenea utile.

Un exemplu de inițializare diferențiată a caracterelor[1]:

```
// .....  
  
/*  
 * Can be optimized later, if statements will do  
 */  
if(this.player_class.equals("Warrior")) { /* DONE */  
    this.hp = 26;  
    this.mp = 4;  
    this.strength = 18;  
    this.agility = 4;  
    this.resilience = 18;  
    this.deftness = 5;  
    this.charm = 4;  
    this.magical_mending = 0;  
    this.magical_might = 0;  
}  
if(this.player_class.equals("Priest")) { /* DONE */  
    this.hp = 19;  
    this.mp = 14;  
    this.strength = 9;  
    this.agility = 14;  
    this.resilience = 9;  
    this.deftness = 9;  
    this.charm = 7;  
    this.magical_mending = 18;  
    this.magical_might = 0;  
}  
// .....
```

Se recomandă totuși utilizarea unor șabloane de proiectare care să ajute la crearea fiecărui caracter în parte cu proprietățile sale.

Caracterul trebuie desenat pe hartă. Pentru aceasta este necesară o funcție care extrage imaginea caracterului și apoi o plasează în poziția precizată.

Observație!

Ipostazele caracterelor trebuie extrase dintr-un spritesheet!!



Pe parcursul jocului personajele pot fi „upgradate”. Pentru aceasta unele atribute se pot modifica pe parcursul jocului în sensul avantajării personajului în urma unor acțiuni precum: trecere la alt nivel, acumularea unor puncte de experiență, acumularea unor obiecte special etc.

În crearea personajelor pot fi utilizate o serie de șabloane de proiectare precum Factory, Abstract Factory etc.

Deplasarea caracterelor

Pentru a controla deplasarea caracterelor în mediul, se poate crea o clasă ControlManager care să gestioneze acțiunile utilizatorului. Va trebui definit încă de la început modul în care personajul este controlat de utilizator(mouse, tastatură, joystick, etc). Evaluarea acțiunii poate genera confuzie, dacă nu este atent implementată.

De cele mai multe ori în această clasă se implementează o metodă de interceptare a acțiunii utilizatorului(“Listener”) și se indică acțiunea dorită. De exemplu apăsarea unor taste ar putea fi tratată astfel:

```
public class KeyManager implements KeyListener
{
    private boolean[] keys;
    /*!< Vector de flaguri pentru toate tastele. Tastele vor fi regasite dupa cod [0 - 255]*/
    public boolean up;      /*!< Flag pentru tasta "sus" apasata.*/
    public boolean down;    /*!< Flag pentru tasta "jos" apasata.*/
    public boolean left;    /*!< Flag pentru tasta "stanga" apasata.*/
    public boolean right;   /*!< Flag pentru tasta "dreapta" apasata.*/

    /*! \fn public KeyManager()
    \brief Constructorul clasei.
    */
    public KeyManager()
    {
        ///Constructie vector de flaguri aferente tastelor.
        keys = new boolean[256];
    }
}
```

```

    }

    public void Update()
    {
        up    = keys[KeyEvent.VK_W];
        down  = keys[KeyEvent.VK_S];
        left  = keys[KeyEvent.VK_A];
        right = keys[KeyEvent.VK_D];
    }
}
// .....

```

Bineînțeles că pot fi introduse diverse taste pentru diferite acțiuni ale personajului/personajelor.

Evident că pentru deplasarea pe o hartă a unui personaj se presupune că la apăsarea unei taste sau a unui buton de mouse, sau la acționarea joystick-ului, etc se modifică coordonatele personajului prin intermediul unei metode specializate, după care trebuie redesenată pozițiile personajelor pe ecran.

O modalitate de a indica noua poziție se poate observa în exemplul de mai jos:

```

/*! \fn public void MoveX()
    \brief Modifica pozitia caracterului pe axa X.
 */
public void MoveX()
{
    //Aduna la pozitia curenta numarul de pixeli cu care trebuie sa se deplaseze pe axa X.
    x += xMove;
}

```

Este necesară însă o verificare asupra depășirii cadrului hărții, camerei, etc pentru a evita “dispariția” caracterului din câmpul visual.

Coliziuni cu mediul sau alte obiecte

În timpul deplasării, personajele pot interacționa cu obiecte fixe sau mobile, pot fi definite spații prin care acestea nu au voie să treacă, sau pot fi lovite de proiectile. Toate acestea se numesc coliziuni. Coliziunile pot fi împărțite în două mari categorii: cele cu mediul, de obicei statice, și cele cu personajele, proiectilele, sau obiectele speciale definite în joc, elemente ce de multe ori sunt mobile pe hartă.

Pentru a trata astfel de coliziuni sunt mai multe metode de abordare:

- Introducerea unor proprietăți de tip SOLID atât personajelor cât și obiectelor sau elementelor de mediu, care să fie de tip boolean și să indice posibilitatea de a trece sau nu prin obiectele respective. Bineînțeles că aceasta presupune o verificare constantă a poziției actuale și a proprietăților obiectelor aflate în același loc.
- O altă metodă ar fi detectarea intersecției dintre dreptunghiul ce definește personajul și obiectele prin care nu se poate trece. Aceasta se poate face prin metode geometrice și determină dacă personajul se suprapune parțial sau total cu pozițiile altor obiecte sau a unor tile-uri prin care nu se poate trece.
- De cele mai multe ori se preferă o metodă hibridă care utilizează ambele metode descrise mai sus.
- Există și alte abordări bazate pe matricea ce indică harta (de exemplu dacă personajul se poziționează pe un tile ce are o valoare anume în matrice ce indică harta se consideră că este un anumit tip de coliziune).

Indiferent de metoda utilizată, tratarea corectă a coliziunilor dă farmec jocului și bineînțeles că trebuie făcută cu multă atenție.

Un exemplu de tratare a coliziunilor se poate vedea mai jos:

```
//.....  
public CollisionChecker(RefLinks reflink){  
    ref = reflink;  
    map = new Map(reflink);  
}  
  
public void checkTile(Hero entity) {  
  
    int entityLeft = (int) (entity.GetX() + entity.normalBounds.x);  
    int entityRight = (int) (entity.GetX() + entity.normalBounds.x + entity.normalBounds.width);  
    int entityUp = (int) (entity.GetY() + entity.normalBounds.y);  
    int entityDown = (int) (entity.GetY() + entity.normalBounds.y + entity.normalBounds.height);  
  
    int entityLeftCol = (int) ((entityLeft - entity.speed) / 32);  
    int entityRightCol = (int) ((entityRight - entity.speed) / 32);  
    int entityUpCol = (int) ((entityUp - entity.speed) / 32);  
    int entityDownCol = (int) ((entityDown - entity.speed) / 32);  
  
    if (ref.GetKeyManager().up) {  
        if (map.GetTile(entityLeftCol, entityUpCol).IsSolid() && map.GetTile(entityRightCol,  
entityUpCol).IsSolid()) {  
  
            entity.collisionOn = true;  
  
        }  
    }  
}  
//.....
```

Bineînțeles că trebuie tratate atât coliziunile cu mediul cât și cu alte personaje sau obiecte. În urma unor astfel de coliziuni personajul poate să acumuleze experiență, să piardă sau crească viața să se modifice scorul sau jocul poate să ia o turnură diferită. **Din acest motiv este recomandat în mod imperativ utilizarea unui șablon de proiectare(de exemplu Observer) care să notifice imediat toate componentele afectate de un astfel de eveniment!**

Câteva cuvinte despre crearea camerei

Există câțiva pași care trebuie realizați când se crează o cameră - prezentarea unei singure secvențe din întreaga imagine a jocului, aceasta mișcându-se de obicei după poziția personajului principal.

Acești pași sunt descriși mai jos:

- Se definește poziția și dimensiunea camerei

Inițial trebuie să se definească locul unde se află la început camera în lumea jocului și care este dimensiunea acesteia. De obicei, poziția camerei este definită ca un vector (matrice, după caz) care reprezintă poziția sa în lumea jocului, iar dimensiunea sa este definită ca un dreptunghi care specifică zona vizibilă din lumea jocului.

- Se configurează matricea de vizualizare (view matrix)

O matrice de vizualizare este o matrice 4x4 care transformă lumea jocului din sistemul său de coordonate global în sistemul de coordonate local al camerei. Pentru a crea o matrice de vizualizare, de obicei trebuie să definiți poziția, orientarea și direcția camerei.

- Aplicarea matricei de vizualizare

În lumea jocului, înainte de a o reda, matricea de vizualizare trebuie creată și aplicată asupra hărții (contextului specific). Acest lucru transformă lumea jocului din sistemul său global de coordonate în sistemul de coordonate local al camerei și se asigură că partea corectă a lumii de joc este vizibilă prin cameră. Sunt necesare metode care reușesc să facă transformări ale coordonatelor între cele două sisteme: a jocului și a camerei.

- Camera în mișcare

Pentru a crea o cameră în mișcare, trebuie să i se actualizeze poziția ca răspuns la intrarea utilizatorului sau a evenimentelor din joc. Acest lucru implică, de obicei, ajustarea vectorului de poziție al camerei în funcție de intrarea utilizatorului și actualizarea matricei de vizualizare.

Este utilă implementarea unei clase sau a unei interfețe Camera. Mai jos (pagina următoare) puteți vizualiza o secvență de cod care implementează acest concept.

Bineînțeles că sunt și alte metode de a implementa o astfel de cameră. Mai mult, ea trebuie să reacționeze și la mișcarea personajelor, și la unele evenimente ce au loc în joc, ceea ce înseamnă că pot fi adăugate și alte metode care să ajute la o astfel de interacțiune.

```

import java.awt.Graphics2D;
import java.awt.Rectangle;
import java.awt.geom.AffineTransform;

public class Camera {
    private Rectangle bounds; // limitele vizibile ale camerei
    private double x; //coordonata x a camerei
    private double y; //coordonata y a camerei
    private double scale; // factorul de scalare al camerei

    // starea camerei; poziție și dimensiune
    public Camera(double x, double y, double width, double height) {
        this.bounds = new Rectangle((int) x, (int) y, (int) width, (int) height);
        this.x = x;
        this.y = y;
        this.scale = 1.0;
    }

    // metoda prin care se setează poziția camerei
    public void setPosition(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // metoda prin care se setează factorul de scalare, daca este nevoie de el
    public void setScale(double scale) {
        this.scale = scale;
    }

    // metoda de aplicare a matricei de vizualizare a camerei
    public void apply(Graphics2D g2d) {
        AffineTransform transform = new AffineTransform();
        transform.translate(-x, -y); // translarea poziției camerei
        transform.scale(scale, scale); // aplicarea factorului de scală
        g2d.setTransform(transform);
    }

    // metoda returnează limitele vizibile ale camerei
    public Rectangle getBounds() {
        return bounds;
    }
}

```

În multe din jocurile actuale o astfel de abordare poate crea suspansul și emoția descoperirii, ceea ce face ca utilizatorii să devină curioși în a explora întreaga lume a jocului. Acest lucru însă poate fi realizat și prin alte metode precum fog of war.

Fog of War

Universul jocului poate descoperit treptat prin acoperirea cu o ceață a majorității suprafeței de joc. Aceasta presupune o zonă vizibilă în jurul eroului/eroilor și în rest o vizibilitate redusă doar asupra unei componente fixe (tiles, mediul, etc) iar alteori acoperirea întregii zone cu o nuanță care nu afișează nimic de pe hartă în afara zonei vizibile din jurul personajului.

Exemple de imagini reprezentând ideea de *fog of war*:



<https://didacromero.github.io/Fog-of-War/>

<https://bit-hack.net/tag/fog-of-war/>

Principalele concepte legate de această tehnică folosită foarte des în jocurile de tip *real time strategy*, poate conține câteva etape precum cele de mai jos:

- Se definește harta jocului

Se crează o matrice bidimensională pentru a reprezenta harta jocului, fiecare element reprezentând o piesă sau o celulă de pe hartă (un tile, etc). Se setează valorile inițiale ale tuturor elementelor pentru a fi ascunse de unitățile jucătorului.

- Se definesc unitățile jucătorului:

Se poate crea o clasă care să reprezinte unitățile jucătorului, inclusiv poziția lor, raza de vizibilitate și raza de mișcare.

- Definirea ceții războiului (fog of war):

Se poate crea o altă matrice bidimensională pentru a reprezenta ceața războiului (fog of war), fiecare element reprezentând vizibilitatea piesei corespunzătoare pe harta jocului. Se setează valorile inițiale ale tuturor elementelor pentru a fi ascunse de unitățile jucătorului.

- Actualizare Fog of War:

Ori de câte ori o unitate/erou al unui jucător se mișcă, trebuie să se actualizeze "fog of war" pentru a dezvălui piesele din raza de acțiune. Se poate utiliza un algoritm

simplu sau unul complex pentru a calcula tile-urile care se află în raza de acțiune a liniei de vedere a unității/eroului.

- Afișarea hărții jocului(randare):

Pentru a randa harta pe baza valorilor matricei Fog of War, plăcile(tiles) care sunt ascunse vederii unităților jucătorului ar trebui să fie redată ca negru sau gri, în timp ce plăcile care sunt vizibile ar trebui să fie redată în culori.

- Fog of War trebuie actualizat în mod continuu:

Fog of War se actualizează la intervale regulate pentru a simula trecerea timpului și pentru a împiedica jucătorul să memoreze pur și simplu aspectul general al hărții.

- Optimizarea Fog of War:

Dacă jocul are o lume de joc mare, se poate lua în considerare optimizarea algoritmului de calculare a zonelor cu “ceață” pentru a evita operațiile inutile. De exemplu, se poate împărți harta jocului în părți mai mici și se pot actualiza doar componentele care sunt afectate de unitățile jucătorului.

Un exemplu de concept pentru o clasă de tip Fog of War poate fi studiată în exemplul următor:

```

public class FogOfWar {
    private boolean[][] visible; // matricea ce păstrează vizibilitatea fiecărei tile
    private int width, height; // dimensiunea hărții
    private int playerX, playerY; // poziția jucătorului- eroului

    public FogOfWar(int width, int height, int playerX, int playerY) {
        this.width = width;
        this.height = height;
        this.playerX = playerX;
        this.playerY = playerY;
        visible = new boolean[width][height];
        updateVisibility();
    }

    // reactualizarea zonei vizibile raportată la noua poziție a jucătorului
    private void updateVisibility() {
        for (int x = 0; x < width; x++) {
            for (int y = 0; y < height; y++) {
                if (distance(playerX, playerY, x, y) <= 5) {
                    visible[x][y] = true;
                } else {
                    visible[x][y] = false;
                }
            }
        }
    }

    // metodă de calculare a zonei vizibile. De exemplu distanța euclidiană
    private double distance(int x1, int y1, int x2, int y2) {
        int dx = x2 - x1;
        int dy = y2 - y1;
        return Math.sqrt(dx * dx + dy * dy);
    }

    // la modificarea poziției jucătorului se reactualizează zona vizibilă
    public void setPlayerPosition(int x, int y) {
        playerX = x;
        playerY = y;
        updateVisibility();
    }

    // verifică dacă un tile este sau nu în zona vizibilă
    public boolean isVisible(int x, int y) {
        return visible[x][y];
    }
}

```

Bibliografie

1. ***, Tiles and tilemaps overview - Game development | MDN (mozilla.org), ultima vizita aprilie 2023.
2. ***, <https://codereview.stackexchange.com/questions/212802/player-class-for-rpg-dnd-type-game>
3. ***, <https://www.cs.rug.nl/search/uploads/Resources/PCI2016TR.pdf>, pag 25
4. ***, https://developer.mozilla.org/en-US/docs/Games/Techniques/2D_collision_detection
5. ***, https://www.youtube.com/watch?v=Ny_YHoTYcxo
6. ***, <https://www.youtube.com/watch?v=vdcOIwkB6dA>
7. ***, <https://docs.oracle.com/en/java/>
8. ***, <https://tilcode.blog/2021/01/09/java-roguelike-notes-5-13-how-to-create-a-field-of-view-or-fog-of-war/>
9. ***, <https://gamedev.stackexchange.com/questions/71024/fog-of-war-is-looping-over-every-tile-inefficient>
10. ***, <https://www.youtube.com/watch?v=pVZglo-BK2c>
11. ***, <https://www.gamedev.net/articles/programming/general-and-gameplay-programming/tilemap-based-game-techniques-handling-terrai-r934/>

Anexa 1

Documentația va fi completată cu:

- Se vor defini clar metodele de joc, de victorie/înfrângere, mutări imposibile, etc.
- Screenshot-uri cu diferite ipostaze din joc.
- Strategia adoptată.
- Structura bazei de date.
- Șabloanele de proiectare utilizate și pentru ce.
- Explicația fiecărui algoritm utilizat, sau a metodelor de interacțiune.
- Diagrame de clase (cele mai importante vor fi explicate).
- Diagrame funcționale.

Se vor explica și elementele de implementare precum:

- **Modelul datelor (Data Design)**
 - prezintă structurile de date importante, formatele fișierelor folosite în cadrul soluției și schema bazei de date.
 - Schema bazei de date este descrisă prin:
 - diagrama schemei bazei de date
 - descrierea semnificației tabelelor și, pentru fiecare tabelă, descrierea semnificației coloanelor și indicarea cheilor primare și referențiale.
- **Modelul arhitectural (Architectural Design)**
 - este o structură ierarhică alcătuită din componente interconectate
 - prezintă arhitectura sistemului – atât descriptiv, cât și sub forma unei diagrame de arhitectură
 - descrie:
 - fiecare componentă a arhitecturii,
 - restricțiile de implementare ale componentelor,
 - interacțiunea dintre componentele sistemului.
 - poate fi reprezentat prin :
 - diagrame de clase, în care relațiile ierarhice se bazează pe generalizare și specializare
- **Modelul interfeței cu utilizatorul (User Interface Design)**
 - prezintă ferestrele aplicației și succesiunea lor în cadrul sistemului.
- **Elementele de testare- dacă s-au efectuat (Testing Issues)**
 - identifică componentele critice ale aplicației (componente a căror performanță influențează decisiv performanța globală a aplicației)
 - propune alternative de proiectare a componentelor critice (pentru a fi folosite în cazul insuccesului soluției propuse inițial).

În notare se va ține cont și de modul în care au fost descrise și implementate elemente precum:

1. Jocul trebuie să aibă minimum **3 niveluri** cu grad **distinct de dificultate și aspect general relativ diferit**.
2. Implementarea coliziunilor.
3. Clase pentru tratarea excepțiilor.
4. Șabloane de proiectare (minim 2 șabloane).
5. Implementarea noțiunii de gaming points (scor).
6. Strategia de joc (multiplayer, single player, arcade etc).
7. Setările jocului să fie citite și scrise din/într-o bază de date SQLite.
8. Implementarea noțiunii de salvare, de încărcare, respectiv de joc nou.
9. Implementare cameră și/sau fog of war (fără proiectul va putea primi cel mult nota 6).
10. Comentarii cod conform unui stil ales (doxygen).
11. Se va realiza documentarea proiectului.
12. Se va puncta și impresia artistică/ originalitatea.

Cerințele obligatorii (fara acestea proiectul NU va fi considerat acceptat):

- Proiectul trebuie să conțină **bază/baze de date** (de preferat se salvează și starea jocului la un moment dat – deci jocul ar trebui să poată fi reluat de unde a rămas)
- Proiectul trebuie să aibă **minim 3 niveluri** distincte (game playing diferit)
- **Harta** trebuie încărcată din baza de date sau din fișier!
- **Proiectul trebuie să treacă de faza de compilare!**
- NU se acceptă decât **jocurile care respectă șablonul dat în laborator**- pot exista ajustări!